

Giải thích chi tiết thuật toán Mo

Nguồn gốc: *MO's Algorithm (Query square root decomposition)*, anudeep2011

data-structure/mo-algorithm-anudeep2011.pdf

Bài viết gốc là *MO's Algorithm (Query square root decomposition)* của anudeep2011, đăng ngày 2014-12-28. Tài liệu này trình bày một chủ đề hữu ích và thú vị nhưng trước đây có khá ít tài nguyên giải thích. Trong các kỳ thi, thuật toán này thường xuất hiện ở các bài Codeforces mức Div. 1 C hoặc D. Một thời gian trước hầu như chưa có bài loại này, nhưng sau đó số lượng bài tăng rất nhanh, và có thể kỳ vọng rằng chúng còn xuất hiện nhiều hơn trong các cuộc thi tương lai.

Contents

1	Mô tả bài toán	1
2	Lời giải đơn giản độ phức tạp $O(N^2)$	2
3	Thuật toán và tính đúng đắn	3
4	Chứng minh độ phức tạp $O(\sqrt{N}N)$	3
5	Phạm vi áp dụng	4
6	Bài tập và mã mẫu	4

1 Mô tả bài toán

Cho một mảng kích thước N , mọi phần tử trong mảng đều không lớn hơn N . Cần trả lời M truy vấn. Mỗi truy vấn có dạng (L, R) ; với truy vấn này, hãy cho biết trong đoạn $[L, R]$ có bao nhiêu giá trị xuất hiện ít nhất 3 lần.

Ví dụ, xét mảng

$$\{1, 2, 3, 1, 1, 2, 1, 2, 3, 1\},$$

với chỉ số bắt đầu từ 0.

Với truy vấn $L = 0, R = 4$, đáp án là 1. Đoạn tương ứng là

$$\{1, 2, 3, 1, 1\},$$

trong đó chỉ có giá trị 1 xuất hiện ít nhất 3 lần.

Với truy vấn $L = 1, R = 8$, đáp án là 2. Đoạn tương ứng là

$$\{2, 3, 1, 1, 2, 1, 2, 3\}.$$

Ở đây giá trị 1 xuất hiện 3 lần và giá trị 2 cũng xuất hiện 3 lần, nên số giá trị xuất hiện ít nhất 3 lần là 2.

2 Lời giải đơn giản độ phức tạp $O(N^2)$

Với mỗi truy vấn, ta duyệt từ L đến R , đếm tần suất xuất hiện của từng phần tử rồi báo đáp án. Khi $M = N$, chương trình sau chạy trong trường hợp xấu nhất với thời gian $O(N^2)$.

```
for each query:
    answer = 0
    count[] = 0
    for i in {l..r}:
        count[array[i]]++
        if count[array[i]] == 3:
            answer++
```

Ta sửa nhẹ thuật toán trên. Độ phức tạp vẫn là $O(N^2)$.

```
add(position):
    count[array[position]]++
    if count[array[position]] == 3:
        answer++

remove(position):
    count[array[position]]--
    if count[array[position]] == 2:
        answer--

currentL = 0
currentR = 0
answer = 0
count[] = 0
for each query:
    // currentL should go to L, currentR should go to R
    while currentL < L:
        remove(currentL)
        currentL++
    while currentL > L:
        add(currentL)
        currentL--
    while currentR < R:
        add(currentR)
        currentR++
    while currentR > R:
        remove(currentR)
        currentR--
    output answer
```

Ban đầu ta luôn duyệt trực tiếp từ L đến R , còn bây giờ ta chỉ điều chỉnh từ vị trí của truy vấn trước sang vị trí của truy vấn hiện tại.

Nếu truy vấn trước là $L = 3, R = 10$, thì khi kết thúc truy vấn đó ta có $currentL = 3$ và $currentR = 10$. Nếu truy vấn tiếp theo là $L = 5, R = 7$, ta di chuyển $currentL$ đến 5 và $currentR$ đến 7.

Hàm `add` nghĩa là thêm phần tử tại vị trí đang xét vào tập hiện tại và cập nhật đáp án tương ứng. Hàm `remove` nghĩa là loại bỏ phần tử tại vị trí đang xét khỏi tập hiện tại và cũng cập nhật đáp án tương ứng.

3 Thuật toán và tính đúng đắn

Thuật toán Mo chỉ thay đổi thứ tự xử lý các truy vấn. Ta có M truy vấn; ta sắp xếp lại chúng theo một thứ tự đặc biệt rồi xử lý. Vì vậy, đây rõ ràng là một thuật toán offline.

Mỗi truy vấn có L và R ; ta gọi chúng lần lượt là **điểm bắt đầu** và **điểm kết thúc**. Chia mảng đầu vào thành $\sqrt{N}$ khối. Khi đó kích thước mỗi khối là

$$\frac{N}{\sqrt{N}} = \sqrt{N}.$$

Mỗi điểm bắt đầu rơi vào một khối nào đó; mỗi điểm kết thúc cũng rơi vào một khối nào đó.

Nếu điểm bắt đầu của một truy vấn nằm trong khối thứ p , ta nói truy vấn đó thuộc về khối thứ p . Thuật toán sẽ xử lý các truy vấn của khối thứ nhất, rồi đến khối thứ hai, cứ như vậy cho đến khối thứ $\sqrt{N}$. Như thế ta đã có một thứ tự: các truy vấn được sắp tăng theo số hiệu khối của điểm bắt đầu. Có thể có nhiều truy vấn thuộc cùng một khối.

Tạm thời bỏ qua các khối khác và chỉ xét cách hỏi, trả lời các truy vấn thuộc khối thứ nhất. Với mọi khối, ta sẽ làm tương tự. Tất cả truy vấn trong khối thứ nhất có điểm bắt đầu thuộc khối thứ nhất, nhưng điểm kết thúc có thể nằm trong bất kỳ khối nào, kể cả chính khối thứ nhất. Bây giờ ta sắp xếp lại các truy vấn này theo thứ tự tăng dần của R . Ta cũng thực hiện thao tác này trong từng khối.

Vậy thứ tự cuối cùng là gì? Tất cả truy vấn trước hết được sắp tăng theo số hiệu khối chứa L . Nếu số hiệu khối bằng nhau, chúng được sắp tăng theo giá trị R .

Ví dụ, xét các truy vấn sau, giả sử có 3 khối kích thước 3, lần lượt là 0–2, 3–5 và 6–8:

$$\{0, 3\} \{1, 7\} \{2, 8\} \{7, 8\} \{4, 8\} \{4, 4\} \{1, 2\}.$$

Trước hết, sắp xếp theo số hiệu khối chứa điểm bắt đầu:

$$\{0, 3\} \{1, 7\} \{2, 8\} \{1, 2\} \mid \{4, 8\} \{4, 4\} \mid \{7, 8\}.$$

Sau đó, trong mỗi khối, sắp xếp theo giá trị R :

$$\{1, 2\} \{0, 3\} \{1, 7\} \{2, 8\} \mid \{4, 4\} \{4, 8\} \mid \{7, 8\}.$$

Bây giờ dùng cùng đoạn mã đã nêu ở phần trước để giải bài toán. Thuật toán đúng vì ta không thay đổi nội dung xử lý của từng truy vấn; ta chỉ sắp xếp lại thứ tự của các truy vấn.

4 Chứng minh độ phức tạp $O(\sqrt{N}N)$

Thuật toán Mo đã hoàn tất: nó chỉ là một cách sắp xếp lại truy vấn. Điểm đáng chú ý nằm ở phân tích thời gian chạy. Nếu xử lý các truy vấn theo thứ tự đã mô tả ở trên, đoạn mã vốn có độ phức tạp $O(N^2)$ sẽ chạy trong thời gian $O(\sqrt{N} \cdot N)$. Chỉ riêng việc sắp xếp lại truy vấn đã giảm độ phức tạp từ $O(N^2)$ xuống $O(\sqrt{N} \cdot N)$ mà không cần sửa thêm logic xử lý truy vấn.

Nhìn vào đoạn mã phía trên, độ phức tạp của toàn bộ truy vấn do bốn vòng lặp while quyết định. Hai vòng lặp đầu có thể được hiểu là tổng quãng đường di chuyển của con trỏ trái `currentL`; hai vòng lặp sau là tổng quãng đường di chuyển của con trỏ phải `currentR`. Tổng của hai đại lượng này là độ phức tạp tổng.

Trước hết xét con trỏ phải. Với mỗi khối, các truy vấn được sắp theo thứ tự tăng của R , nên `currentR` di chuyển theo hướng tăng. Khi chuyển sang khối tiếp theo, con trỏ có thể đang ở tận đầu mút bên phải

và phải di chuyển đến giá trị R nhỏ nhất trong khối kế tiếp. Như vậy, với một khối cố định, lượng di chuyển của con trỏ phải là $O(N)$. Có $O(\sqrt{N})$ khối, nên tổng lượng di chuyển là

$$O(N\sqrt{N}).$$

Tiếp theo xét con trỏ trái. Với mỗi khối, điểm bắt đầu L của tất cả truy vấn đều nằm trong cùng khối. Khi chuyển từ một truy vấn sang truy vấn khác, con trỏ trái có thể di chuyển, nhưng vì L cũ và L mới nằm trong cùng một khối, quãng đường di chuyển là $O(\sqrt{N})$, tức kích thước khối. Nếu trong khối đó có Q truy vấn, tổng lượng di chuyển của con trỏ trái trong khối là $O(Q\sqrt{N})$. Tính trên mọi khối, tổng độ phức tạp phần này là $O(M\sqrt{N})$.

Vì vậy, tổng độ phức tạp là

$$O((N + M)\sqrt{N}).$$

Trong trường hợp thường xét $M = N$, ta thu được

$$O(N\sqrt{N}).$$

5 Phạm vi áp dụng

Như đã nói ở trên, thuật toán này là offline. Điều đó nghĩa là nếu đề bài bắt buộc phải trả lời truy vấn đúng theo thứ tự xuất hiện, ta không thể dùng trực tiếp thuật toán này. Điều này cũng có nghĩa là khi có thao tác cập nhật, thuật toán ở dạng trên không dùng được.

Không chỉ vậy, còn có một giới hạn quan trọng: ta phải viết được hai hàm add và remove. Trong nhiều trường hợp, add rất đơn giản, nhưng remove thì không. Một ví dụ là bài toán hỏi giá trị lớn nhất trong đoạn. Khi thêm một phần tử, ta có thể dễ dàng theo dõi giá trị lớn nhất; nhưng khi xóa một phần tử, việc duy trì giá trị lớn nhất không còn hiển nhiên. Trong tình huống đó, ta có thể dùng một tập hợp để thêm phần tử, xóa phần tử và báo giá trị cần tìm. Khi ấy mỗi thao tác thêm và xóa tốn $O(\log N)$, dẫn đến thuật toán $O(N\sqrt{N}\log N)$.

Trong rất nhiều bài toán ta có thể dùng thuật toán này. Một số bài cũng có thể giải bằng cấu trúc dữ liệu khác như cây đoạn, nhưng có những bài mà thuật toán Mo là cách tiếp cận then chốt.

6 Bài tập và mã mẫu

- **DQUERY – SPOJ.** Số phần tử khác nhau trong đoạn chính là số phần tử có số lần xuất hiện ít nhất 1, nên bài toán tương tự với bài đã thảo luận ở trên.
- **Powerful Array – Codeforces Div. 1 D.** Đây là một ví dụ mà thuật toán Mo rất phù hợp. Chỉ cần sửa hai hàm `add()` và `remove()` trong đoạn mã phía trên.
- **GERALD07 – CodeChef.**
- **GERALD3 – CodeChef.**
- **Tree and Queries – Codeforces Div. 1 D.**
- **Powerful Array – Codeforces Div. 1 D.**
- **Jeff and Removing Periods – Codeforces Div. 1 D.**
- **Sherlock and Inversions – CodeChef.**

Tài liệu nguồn có nhắc đến mã mẫu liên kết bên ngoài và ghi chú rằng mã mẫu đó có thể bị quá thời gian nếu không dùng nhập xuất nhanh; để giữ mã gọn, phần nhập xuất nhanh đã được lược bỏ. Một bình luận cũng cho rằng khai báo `remove` và `add` là `inline` có thể đủ để được chấp nhận.